SYSTEM ANALYSIS DOCUMENTATION (SAD)

UMHackathon 2026

TEAM ID: 128

TEAM NAME: NULL

---

1. Introduction 3
1.1. Purpose 3
1.2. Background 4
1.3. Target Stakeholders 5
2. System Architecture & Design 6
2.1. High-Level Architecture 6
2.1.1. Overview 6
2.1.2. LLM as Service Layer 6
2.1.3. Dependency Diagram 7
2.1.4. Sequence Diagram 7
2.2. Technological Stack 7
2.2.1. (Frontend) 7
2.2.3. Database: MySQL 7
2.2.4. (Cloud/Deployment):GCP Compute Engine (Server), Cloud SQL (MySQL), Firebase (Push Notifications for price alerts) 7
2.2.5. Key Data Flows 7
2.2.6. (do show the how data are moving through the system and how its structure and stored) 7
2.2.7. Normalized Database Schema 8
3. Functional Requirements & Scope 8
3.1. Minimum Viable Product 8
3.2. Non-Functional Requirements (NFRs) 8
3.3. Out of Scope / Future Enhancements 8
4. Monitor, Evaluation, Assumptions & Dependencies 8
4.1. Technical Evaluation: 8
4.1.1. Grayscale Rollout & A/B Testing 8
4.1.2. Emergency Rollback (ER) & Golden Release 8
4.1.3. Priority Matrix 8
4.1.3.1. (Clearly define the Priority Matrix and what are the trigger conditions and what actions will be taken for your project) 8
4.2. Monitoring 8
4.2.1. (Show the planning for your monitoring plan of the system and how it will trigger if any damage is detected of the health of the system.) 8
4.3. Assumptions 8
4.4. External Dependencies 8
5. Project Management & Team Contributions 8
5.1. Project Timeline 8
5.2. Team Members Role 8
5.3. Recommendation 8

1. Introduction

Most SME owners manage their shops using "gut feeling," which leads to two main risks: stockouts (losing customers) or overstocking (trapping cash in unsold items). While big brands use expensive data teams, local sellers struggle to adjust their prices when costs rise or market demand shifts.

The EZ System is a smart assistant designed to level the playing field. It connects a shop's internal records with market trends. By using the Z.AI GLM-5.1 model, the system analyzes sales patterns and suggests the best time to restock or change prices, helping SMEs stay profitable and competitive.

1.1. Purpose

This System Analysis and Design (SAD) document outlines the technical structure and design choices behind this EZ System. It serves as a blueprint for how AI and real-world data work together to help local shop owners make better business decisions.

Architecture: The system uses a Three-Layer Architecture. It consists of a Data Tier (MySQL) for storing sales and stock records, a Logic Tier (Python) to clean and prepare data, and an Intelligence Tier (Z.AI GLM-5.1) that acts as the "brain" for making complex decisions.

Data Flows: The flow begins with internal shop data (daily sales and inventory levels) moving from the database to the backend. The system is designed to eventually pull in external data—like public holiday calendars or price hike news—to give better context to its advice.

Model Process: The model shows the end-to-end workflow of the core features: ingesting Shopee seller metrics and scanning business health to analyze margin risks against external triggers, such as global price fluctuations or local public holidays. It specifically models the "Thinking Mode" of the GLM-5.1 to perform Trade-Off analysis. This ensures that every specific action, such as "Buy stock now" or "Increase price by 5%", is backed by a verifiable chain of reasoning that the shop owner can trust and verify.

Role of Reference: This document serves as the primary guide for developers and testers. It ensures that the high-level AI logic is correctly built into the app and provides a standard for testing the "Integrity Shield" to keep the business data safe and accurate.

1.2. Background

Many local Malaysian SMEs have successfully moved their businesses onto online platform. However,

while selling online is easy, managing the business behind the scenes is still very manual. Currently, shop owners have to manually check their stock, manually calculate their profits, and manually keep track of news (like tax changes or price hikes) to decide if they should change their prices.

Previous Version: Existing tools for small sellers are usually simple dashboards or basic spreadsheets. These tools are "disconnected." A seller might use one app to see sales, another app to check their bank balance, and read the news on social media. There is no single system that connects all these dots to tell the owner exactly what to do next.

Major Changes: This project introduces three major shifts in how SME software works:

- **Active Reasoning:** Instead of just showing a graph of past sales, the system uses "Thinking Mode" to suggest future actions.
- **External Data Integration:** The system "listens" to the real world (news, taxes, holidays) and combines it with the shop's private data.
- **Integrity Shield:** A new security layer that ensures the AI is making decisions based on clean, honest data, preventing errors or "hacked" inventory numbers.

1.3. Target Stakeholders

Stakeholders	**Roles**	**Expectations**
SME Owner (Shop Owner)	Uses the app to check business health and get advice on pricing and restocking.	Simple advice that increases profit, saves time, and prevents them from running out of cash.
Development Team	Builds the Flutter app, Python backend, and connects the Z.AI GLM-5.1 model.	A clear plan for how the AI should "think" and modular code that is easy to update.
QA Team (Testers)	Tests the system by entering "bad" or "fake" data to see if the AI detects errors.	Clear rules on what counts as "wrong" data and a stable environment to test the "Integrity Shield."
Project Judges	Review the project to see if it solves a real problem using smart technology.	A logical connection between AI "Thinking Mode" and the economic empowerment of small businesses.
Suppliers (Indirect)	Provide the physical stock that the SME owner needs to buy.	More consistent and predictable orders from the shop owner due to better inventory planning.

2. System Architecture & Design

2.1. High-Level Architecture

2.1.1. Overview

Type	Details
System	Mobile Application (Flutter - Android/ iOS)
Architecture	Cloud-Native Monolithic-Core with AI Service Layer

The EZ System is structured as a client-server-based architecture where the Flutter mobile application serves as the primary user interface for SME owners to manage products, view analytics, and receive AI-driven recommendations. The application communicates with a backend built using FastAPI through a RESTful API layer. The backend follows a cloud-native monolithic-core design with modular components, consisting of a Product Management Service (handling full CRUD operations for products and inventory), an Analytics Service (processing business metrics such as revenue, profit, and stock trends), and an AI Decision Orchestrator powered by Z.AI's GLM, which performs context-aware reasoning, trade-off analysis, forecasting, and recommendation generation. All modules interact with a centralized MySQL database managed via MySQLWorkbench, which stores product data, sales records, user profiles, and AI-generated decision logs. The system is deployed in a cloud environment to ensure scalability and real-time processing, where all decision-making capabilities are fully dependent on the GLM layer, ensuring that without the AI component, the system cannot generate meaningful insights or optimized business decisions.

2.1.2. LLM as Service Layer

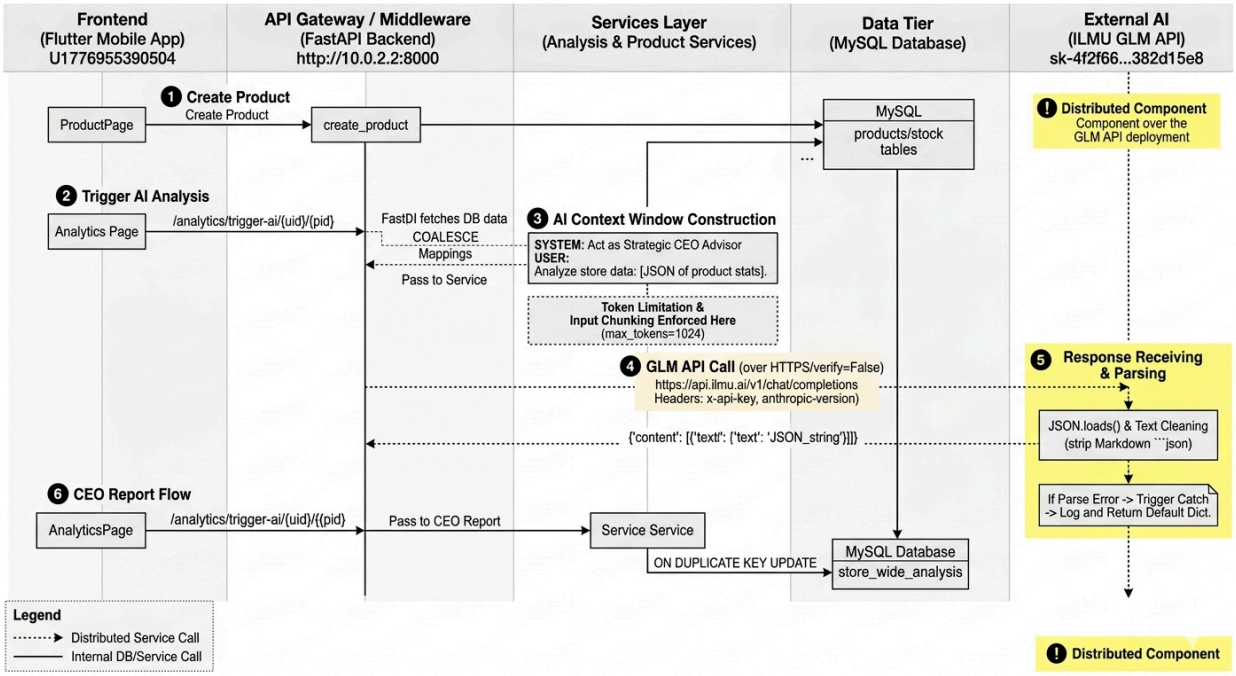
The architecture utilizes Z.AI GLM-5.1 as a specialized reasoning engine rather than a generic chatbot. Its integration into the system follows this logic:

- Prompt Construction: The application acts as the data collection interface across the Home Dashboard, Product Management, Analytics, Alerts, and Profile pages, gathering structured business data such as revenue, profit, stock levels, and product performance from the database. The Analytics module serves as the Decision Orchestrator by combining this internal business data with operational constraints (cost, inventory status, demand trends) into a structured prompt formatted as: [Business Metrics] + [Operational Constraints] + [Required Decision Task] = Actionable Business Recommendation.
- Context Window Management: To ensure efficient processing, only relevant snapshots such as current stock levels, recent sales performance, and key KPIs are sent to GLM, while historical

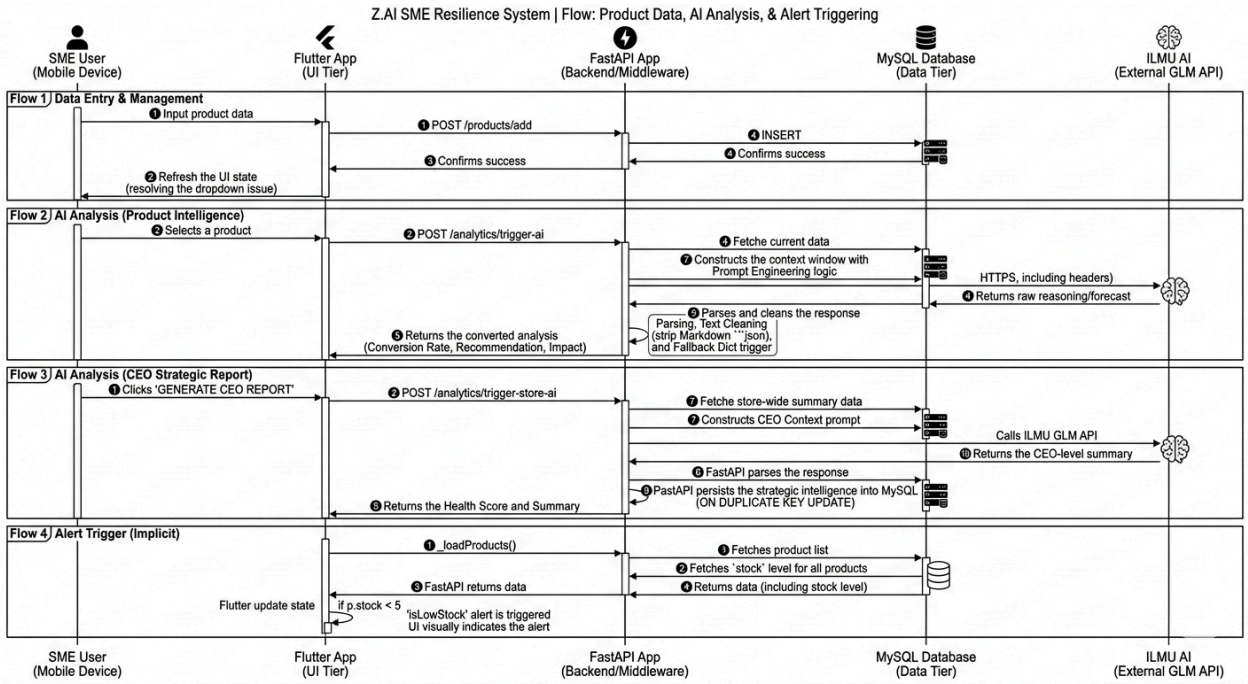
records are aggregated and summarized in the backend database. This prevents context overload and ensures the model focuses only on decision-critical information.

- Response Parsing: The system enforces structured output from GLM in a fixed JSON format containing recommendations, forecasts, and explanations. The backend validates this output against predefined schemas (e.g., ensuring no invalid pricing or missing fields) before rendering the results in Flutter as decision cards, charts, and alerts.
- Token Enforcement: A lightweight API control layer manages request size by monitoring input data volume from the Flutter app. If a dataset exceeds the allowed threshold (e.g., too many products or extended sales history), the system automatically batches or filters the data by category (e.g., high-priority products first) before sending it to GLM, ensuring stable performance and cost-efficient AI processing.

2.1.3. Dependency Diagram



2.1.4. Sequence Diagram



2.2. *Technological Stack*

2.2.1. Frontend: Flutter [Cross-platform mobile framework used to build a single code base for iOS and Android]

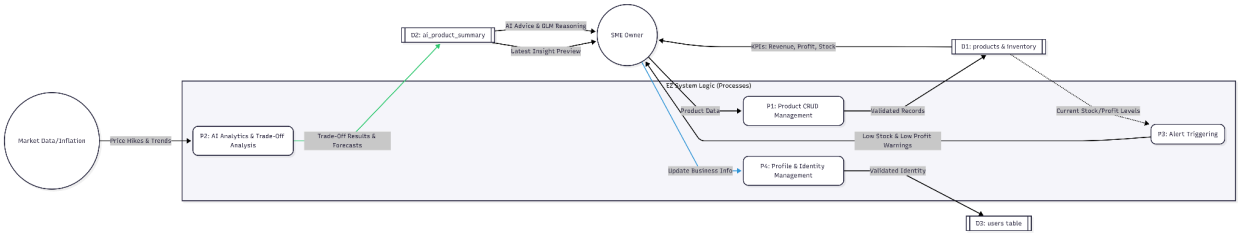
2.2.2. Backend: FastAPI (Python) [Chosen for high performance and native support for asynchronous AI model Calling]

2.2.3. Database: MySQL (MySQL Workbench) [Relational database used to store all business data including products, stock levels, user profiles, and AI-generated insights for decision tracking and analytics]

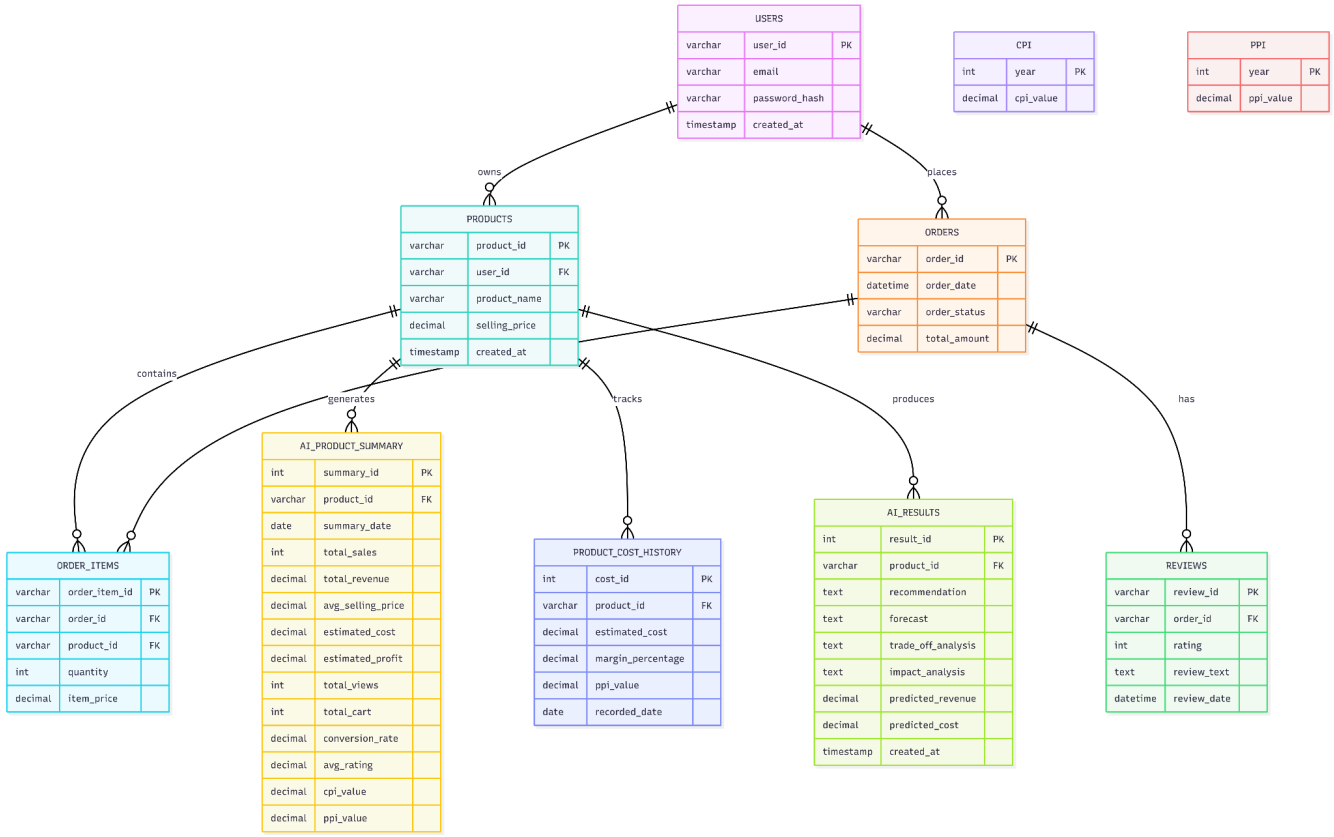
2.2.4. Cloud/Deployment: Local Server [Hosts the FastAPI backend and MySQL database, enabling secure API communication between Flutter frontend and the GLM service layer for real-time decision intelligence processing]

2.3. *Key Data Flows*

2.3.1. Data Flow Diagram (DFD)



2.3.2. Normalized Database Schema



3. *Functional Requirements & Scope*

The project focuses on building a high-impact Minimum Viable Product (MVP) that demonstrates how AI can help SME e-commerce sellers make better pricing and inventory decisions based on their sales data and market conditions.

3.1. Minimum Viable Product

Features	Description
Intelligent Sales Data Ingestion	Sellers input sales data (email, password, business name, product name, cost, selling price, stock quantity, units sold). The system validates and verifies the data input before storing it in the database (users, products, inventory). A summarized result (ai_product_summary) is generated for AI processing.

Business Health Dashboard	The system presents a dashboard that displays key business metrics such as total revenue, profit, number of products, and inventory status. It also highlights performance trends and provides AI-driven insights, allowing sellers to quickly understand their current business condition and make informed decisions.
AI Pricing & Inventory Recommendation	Using GLM-5.1, the system analyzes sales data, cost history, and inflation (CPI data) to recommend optimal product pricing and restock quantities. All recommendations are stored in the ai_results table.
Interactive AI	Sellers can input a hypothetical price (e.g. RM20), and the system predicts the impact on demand, revenue, and profit using AI reasoning. This helps users evaluate decisions before applying them.
Explainability & Trade-off Analysis	For every AI output, the system provides a structured explanation including recommendation reasoning, forecast, trade-off analysis, and impact analysis (time, cost, revenue). This improves trust and decision transparency.

3.2. Non-Functional Requirements (NFRs)

Quality	Requirements	Implementation
Data Integrity	The system must prevent invalid or “dirty” data such as negative stock, incorrect prices, or missing values from affecting AI results.	Backend validation module (Integrity Shield) ensures all uploaded CSV data is cleaned before insertion into MySQL tables.
Decision Reliability	AI outputs must remain realistic and within a reasonable business range.	A guardrail system ensures recommendations stay within ±30% of current price or cost benchmarks before saving into ai_results.
Performance & Token Latency	AI responses must be fast enough to support real-time interaction in the simulator.	Async API calls with a 5-second timeout and a user-facing “Thinking Mode” loading state.

Scalability	The system should support multiple SME users and increase sales data volume.	Use modular backend design (FastAPI services) and optimized database queries with indexing on product_id and user_id.
Cost Efficiency	AI processing cost per user must remain low for SME accessibility.	Prompt chunking for large datasets and caching of CPI/market data for 24 hours to reduce repeated API calls.
Security & Privacy	Business data must be protected from unauthorized access or leaks.	AES-256 encryption for sensitive data and anonymization of user-related information before AI processing.
Maintainability	The system should be easy to update and extend with new AI features.	Separation of modules (data ingestion, AI engine, database layer) and clean API structure for future enhancements.

3.3. Out of Scope / Future Enhancements

The following features are part of the long-term vision but will not be implemented during UMHackathon 2026:

- Automated Supplier Procurement
  The system automatically places restock orders with suppliers based on AI recommendations.
- Predictive Cash Flow Financing
  AI evaluates seller performance and suggests loan eligibility or credit support.
- Omnichannel Integration
  Integration with platforms such as TikTok Shop, Lazada, and physical POS systems for a unified sales dashboard.
- Real-Time Competitor Price Tracking
  Monitoring competitor prices dynamically and generating counter-pricing strategies.
- AI-Generated Marketing Content
  Automatically generating promotional banners, product descriptions, and social media content.

4. *Monitor, Evaluation, Assumptions & Dependencies*

4.1. Technical Evaluation:

4.1.1. Grayscale Rollout & A/B Testing

Strategy: Controlled Staged Deployment.
The SME AI system uses a staged rollout approach where updates to the rule_engine logic and LLM prompt structure are first deployed to a small group of internal users (approximately 5–10%) before full production release. This allows monitoring of AI-generated business insights (recommendation, trade-off analysis, impact analysis) without affecting all users in the system.

A/B Testing Model:
Group A (Control) receives rule-based output only from the rule_engine() without LLM enhancement, while Group B (Test) receives LLM-enhanced JSON recommendations generated through the llm_engine() using structured KPI data. This comparison helps evaluate the added value of AI reasoning versus deterministic rule-based analysis.

Success Metrics:
System performance is measured using recommendation accuracy consistency (alignment with KPI values such as profit and conversion rate), stability of JSON output format, and user interaction feedback (e.g., acceptance of AI recommendations). A successful deployment requires that AI outputs remain consistent with rule-based logic and do not introduce invalid or hallucinated business insights.

4.1.2. Emergency Rollback (ER) & Golden Release

Golden Release: Version 1.0.0 is defined as the stable production baseline acting as the system's "Safety Anchor" for the SME AI platform.

Trigger Conditions for Emergency Rollback (ER):
An emergency rollback is triggered if the AI enters a recursive or unstable reasoning state, identified by unusually high processing latency (greater than 15 seconds per inference cycle). Additionally, if the system's rule-based safety layer (rule_engine) fails to correctly flag a loss-generating recommendation as "unsafe," it is considered a Shield Breach, indicating a critical logic failure in business constraint validation.

Action Mechanism:
When either condition occurs, the CI/CD pipeline automatically initiates a rollback to the last stable container image of Version 1.0.0. This ensures system

stability, prevents propagation of incorrect financial recommendations, and maintains zero downtime for SME users during recovery.

4.1.3. Priority Matrix

Severity Level	Trigger Condition	System Action
P1: Critical	Database connection failure OR Integrity/Rule Engine logic failure causing incorrect KPI or financial output.	Immediate system lockdown. Disable all AI-driven price updates and analytics. Activate Maintenance Mode UI to prevent user interaction with unstable data.
P2: High	Z.AI / GLM API timeout, failure, or HTTP 500 server error.	Trigger failover mechanism. Display fallback message such as "AI Insights Temporarily Unavailable." Serve last cached rule-based recommendations from `rule_engine`.
P3: Medium	Dashboard visualization failure (e.g., charts not rendering KPI trends correctly).	Log error into `tbl_system_logs`. Automatically refresh frontend state in Flutter dashboard without affecting backend processing.
P4: Low	Minor UI issues such as layout overflow, spacing misalignment, or non-critical logging delay.	No system interruption. Issue is queued and scheduled for resolution in the next maintenance or patch cycle.

4.2. Monitoring

Health Check (Heartbeat):

The SME AI backend performs a scheduled heartbeat check every 60 seconds to ensure continuous connectivity between the Python FastAPI service, MySQL database, and Z.AI/GLM API. If any service becomes unreachable, the system flags the component as degraded and triggers fallback mechanisms to maintain system stability.

Integrity Damage Detection:

The system continuously monitors data integrity to detect unauthorized or inconsistent modifications, particularly in pricing and KPI-related fields. If a product price is updated without a corresponding entry in the audit ledger or database transaction log, the system immediately triggers a security alert and flags the record as potentially tampered, preventing unreliable financial analysis.

AI Confidence Monitoring:

The system evaluates the confidence level of responses returned by Z.AI/GLM. If the confidence score falls below 50%, the AI-generated recommendation is automatically suppressed and not shown to the user. Instead, the system prompts for additional or more complete input data to ensure that only reliable and high-confidence business insights are presented.

4.3. Assumptions

Data Honesty:

The system assumes that SMEs provide accurate and reliable base cost data, which is essential for the Integrity Shield to correctly compute profit, loss, and pricing recommendations. Any incorrect or manipulated input data may lead to inaccurate AI-driven business insights.

Network Stability:

The system assumes stable and continuous HTTPS connectivity to external services, including Z.AI/GLM APIs and market signal sources. Any network disruption may affect the retrieval of real-time data and result in fallback to cached or rule-based recommendations.

4.4. External Dependencies

Z.AI GLM-5.1 Engine: The primary AI reasoning component used for generating strategic business insights, recommendations, and KPI-based analysis within the SME system.

Flutter SDK: The cross-platform development framework used to build the SME dashboard interface, enabling consistent user experience across web and mobile applications.

5. Project Management & Team Contributions

5.1. Project Timeline

Day 1–2: 17/4 (Friday) – Project Definition & Alignment

- Finalize project idea and direction
- Define core problem statement and target users (SME sellers)
- Identify and lock **2–3 core features only**
- Output: Final system concept + feature scope definition

Day 3: 18/4 (Saturday) – System Design & Documentation

- Start PRD documentation and system analysis
- Design system architecture (AI + DB + GLM flow)
- Create UI wireframes
- Output: Draft PRD + architecture diagram + wireframes

Day 4: 19/4 (Sunday) – Foundation Development

- Build frontend UI structure
- Initialize backend setup and database schema
- Establish project repository structure
- Output: Basic UI visible + backend skeleton ready

Day 5: 20/4 (Monday) – System Integration Phase 1

- Connect frontend with backend APIs
- Implement initial data flow between UI and database
- Begin GLM API integration setup
- Output: Partial working system with data flow

Day 6: 21/4 (Tuesday) – Core Feature Development

- Implement main AI decision engine (pricing + inventory logic)
- Improve GLM prompting and reasoning pipeline
- Output: Core business logic functional (AI layer prepared, API not yet active)

Day 7: 22/4 (Wednesday) – Prototype Completion

- Integrate all modules into one system
- Finalize end-to-end workflow
- Output: Fully working prototype (MVP complete)

Day 8: 23/4 (Thursday) – Testing & Stabilization

- Receive and integrate Z.AI GLM API key
- Connect GLM service layer into Analytics module
- Test prompt-response flow and fix integration issues
- Validate structured output format (JSON responses)
- Output: AI-powered system activated (GLM fully integrated)

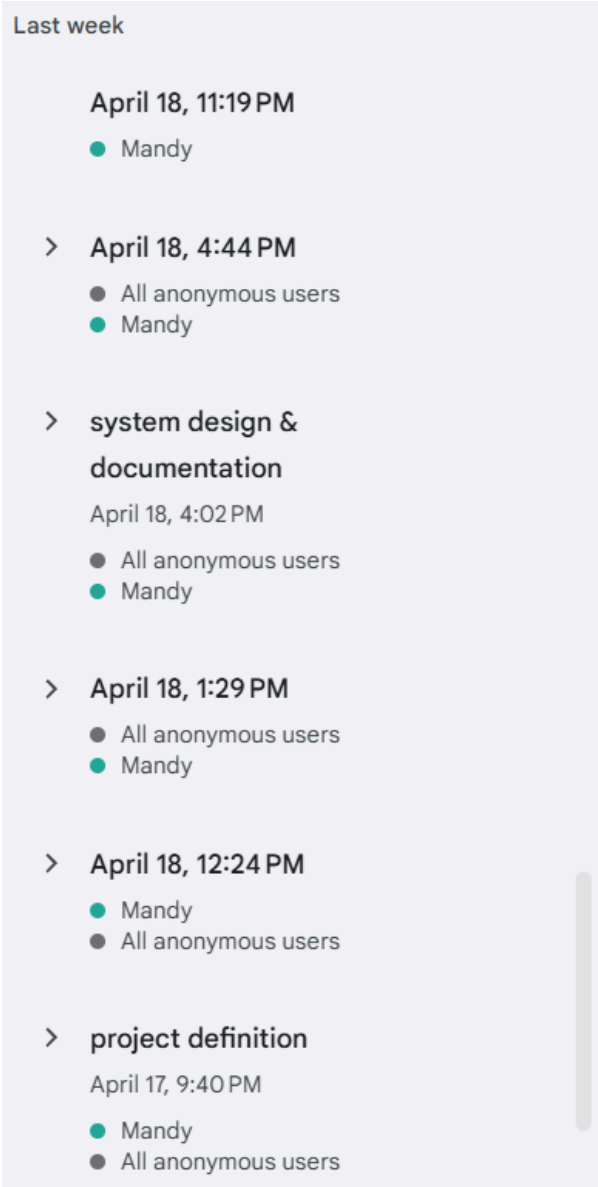
Day 9: 24/4 (Friday) – Documentation Finalization

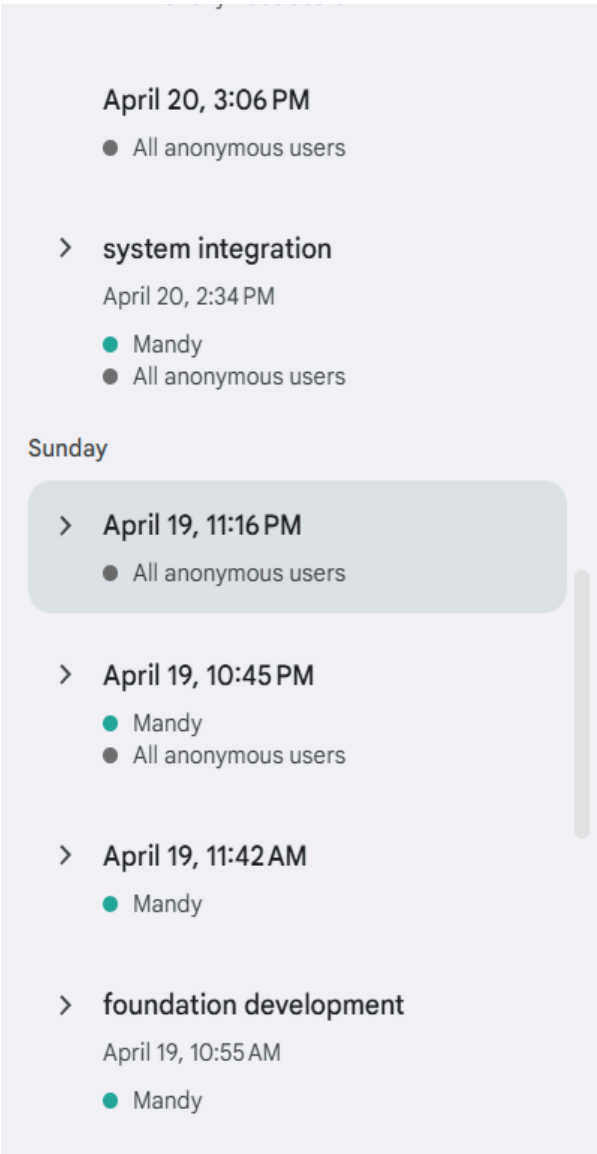
- Conduct full system QA testing
- Fix bugs in AI response handling and UI display
- Improve prompt accuracy and response consistency
- Simulate real SME decision scenarios
- Output: Stable, tested AI decision intelligence system

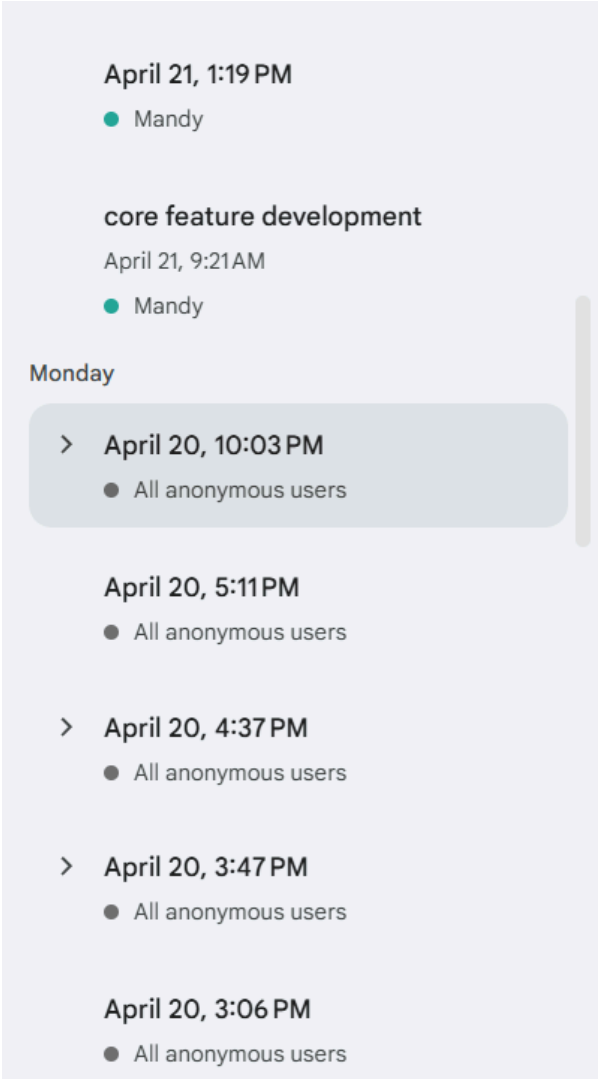
Day 10: 25/4 (Saturday) – Pitch & Presentation Preparation

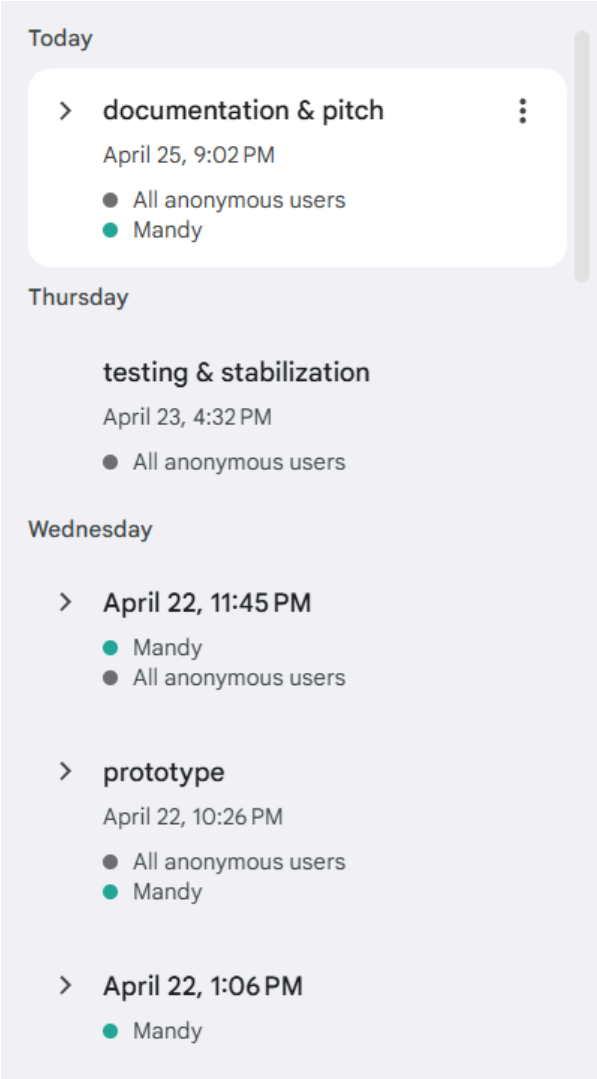
- Finalize PRD and system documentation (~90%)
- Refine architecture diagrams and technical explanation
- Record demo video with full workflow (AI + system)
- Prepare pitch deck slides
- Output: Pitch video + presentation deck completed

Screenshots of Google Docs (Timeline)









5.2. Team Members Role

P1: Lead + Integration + Pitch - CHUAH YAN SHEN
P2: Frontend (UI) - LIM JIA YING
P3: Frontend (Logic) - TAN TONG CHONG
P4: Backend (Database/Core) - LIM ZHI YING
P5: Backend (API/Integration) - OOI WEI JIN

5.3. Recommendation

Scalability

- Use modular architecture (separate AI, backend, frontend layers)
- Cache repeated GLM queries to reduce API cost and latency
- Design database with indexed tables for fast retrieval

Performance

- Implement data preprocessing (chunking + summarization) before sending to GLM
- Limit token usage by filtering irrelevant data early
- Use asynchronous API calls for faster UI response

Reliability

- Add fallback mechanism when GLM fails or returns inconsistent outputs
- Validate AI outputs against database constraints before display
- Ensure user confirmation is required before applying any decision